



**III Year I Semester  
KR24  
ARTIFICIAL INTELLIGENCE  
Department of CSE**

**UNIT-I PART-1 NOTES**

**Unit-I:**

**Introduction to Deep Learning: Fundamentals of Neural Networks and Deep Learning, Deep Learning Applications, Deep Learning Framework**

**Regression:** Neuron Model, Optimization Method, Linear Model in Action.

**Classification:** Hand Written Digital Picture Dataset, Build a model, Error Calculation, Non-Linear model, model complexity, Optimization Method, Hands-On Hand-Written Digital Image Recognition.

**CONTENTS**

**UNIT 1: Introduction to Deep Learning**

**Session 1: Introduction to Artificial Intelligence, Machine Learning, and Deep Learning**

- 1.1 Artificial Intelligence (AI)
- 1.2 Machine Learning (ML)
- 1.3 Deep Learning (DL)
- 1.4 Evolution of AI, ML, and DL
- 1.5 AI vs ML vs DL
- 1.6 Applications of Deep Learning
- 1.7 Advantages and Limitations of Deep Learning
- 1.8 Deep Learning Workflow
- 1.9 Summary
- 1.10 Viva Questions

**Session 2: Fundamentals of Neural Networks**

- 2.1 Introduction to Neural Networks
- 2.2 Biological Neuron vs Artificial Neuron
- 2.3 Components of an Artificial Neuron
- 2.4 Mathematical Model of a Neuron
- 2.5 McCulloch–Pitts Neuron Model
- 2.6 Perceptron Model
- 2.7 Single-Layer Perceptron (SLP)
- 2.8 Multi-Layer Perceptron (MLP)
- 2.9 Neural Network Architecture
- 2.10 Forward Propagation
- 2.11 Advantages and Limitations of Neural Networks

- 2.12 Summary
- 2.13 Viva Questions

### **Session 3: Activation Functions**

- 3.1 Introduction to Activation Functions
- 3.2 Need for Activation Functions
- 3.3 Binary Step Function
- 3.4 Linear Activation Function
- 3.5 Sigmoid Function
- 3.6 Hyperbolic Tangent (Tanh) Function
- 3.7 Rectified Linear Unit (ReLU)
- 3.8 Leaky ReLU
- 3.9 Exponential Linear Unit (ELU)
- 3.10 Softmax Function
- 3.11 Comparison of Activation Functions
- 3.12 Selection of Activation Functions for Different Problems
- 3.13 Summary
- 3.14 Viva Questions

### **Session 4: Loss Functions, Gradient Descent, Optimization Methods, and Backpropagation**

- 4.1 Introduction to Model Training
- 4.2 Loss Function and Cost Function
- 4.3 Mean Squared Error (MSE)
- 4.4 Binary Cross-Entropy Loss
- 4.5 Optimization in Deep Learning
- 4.6 Gradient Descent Algorithm
- 4.7 Learning Rate
- 4.8 Batch Gradient Descent
- 4.9 Stochastic Gradient Descent (SGD)
- 4.10 Mini-Batch Gradient Descent
- 4.11 Backpropagation Algorithm
- 4.12 Numerical Example of Weight Update
- 4.13 Complete Training Workflow
- 4.14 Summary
- 4.15 Viva Questions

### **Session 5: Deep Learning Frameworks and Complete Deep Learning Workflow**

- 5.1 Introduction to Deep Learning Frameworks
- 5.2 Features of Deep Learning Frameworks
- 5.3 TensorFlow

- 5.4 Keras
- 5.5 PyTorch
- 5.6 JAX
- 5.7 Comparison of Deep Learning Frameworks
- 5.8 Complete Deep Learning Workflow
- 5.9 Real-World Case Study
- 5.10 Advantages and Limitations of Deep Learning Frameworks
- 5.11 Summary
- 5.12 Viva Questions
- 5.13 Examination Questions

**Prepared by**

M. ASHA

Assistant Professor

Department of CSE (AI & ML)

# Unit 1: INTRODUCTION TO DEEP LEARNING

## Session 1: Introduction to Artificial Intelligence, Machine Learning and Deep Learning

### Learning Objectives

- Define Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL).
- Understand the evolution of AI.
- Differentiate AI, ML, and DL.
- Explain the need for Deep Learning.
- Identify real-world applications of Deep Learning.
- Understand the advantages and limitations of Deep Learning.

### 1. What is Artificial Intelligence (AI)?

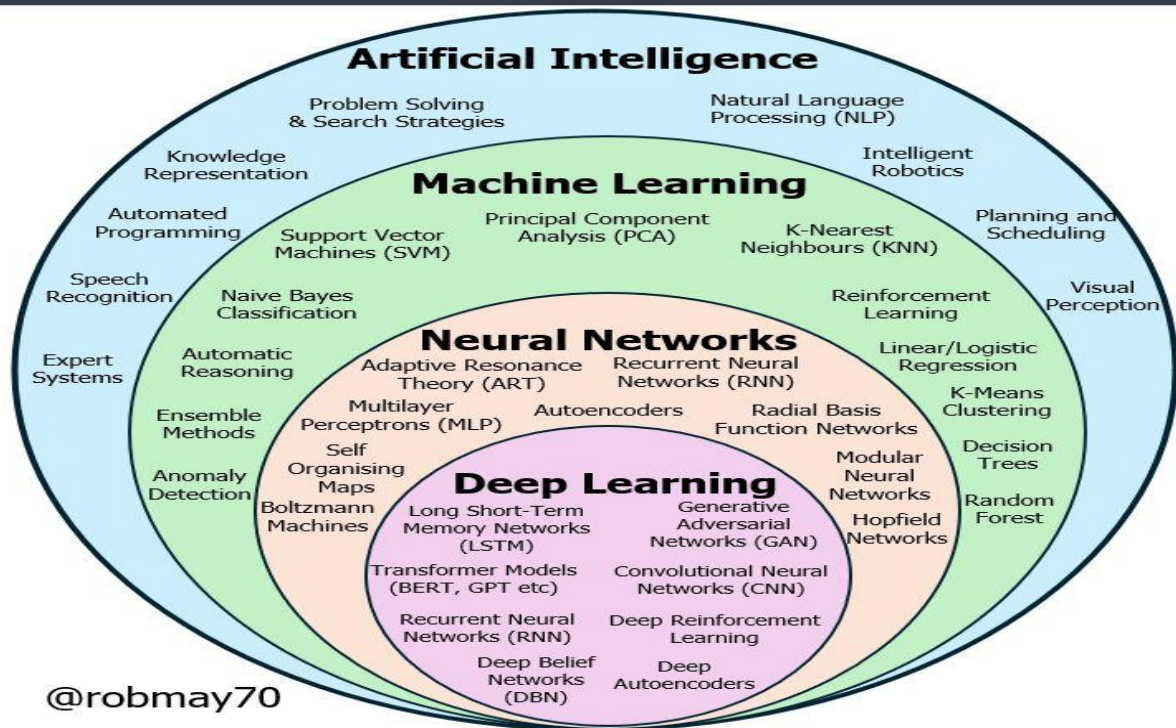
#### Definition

**Artificial Intelligence (AI)** is the branch of computer science that enables machines to mimic human intelligence. AI systems can perform tasks that normally require human intelligence such as learning, reasoning, decision-making, language understanding, and perception.

#### Simple Definition

Artificial Intelligence is the ability of a computer or machine to think and act intelligently.

# AI Core Components



## Examples

- Google Assistant
- Siri
- Alexa
- Self-driving cars
- Face Recognition
- Chess Playing Computers
- Medical Diagnosis Systems

## Characteristics of AI

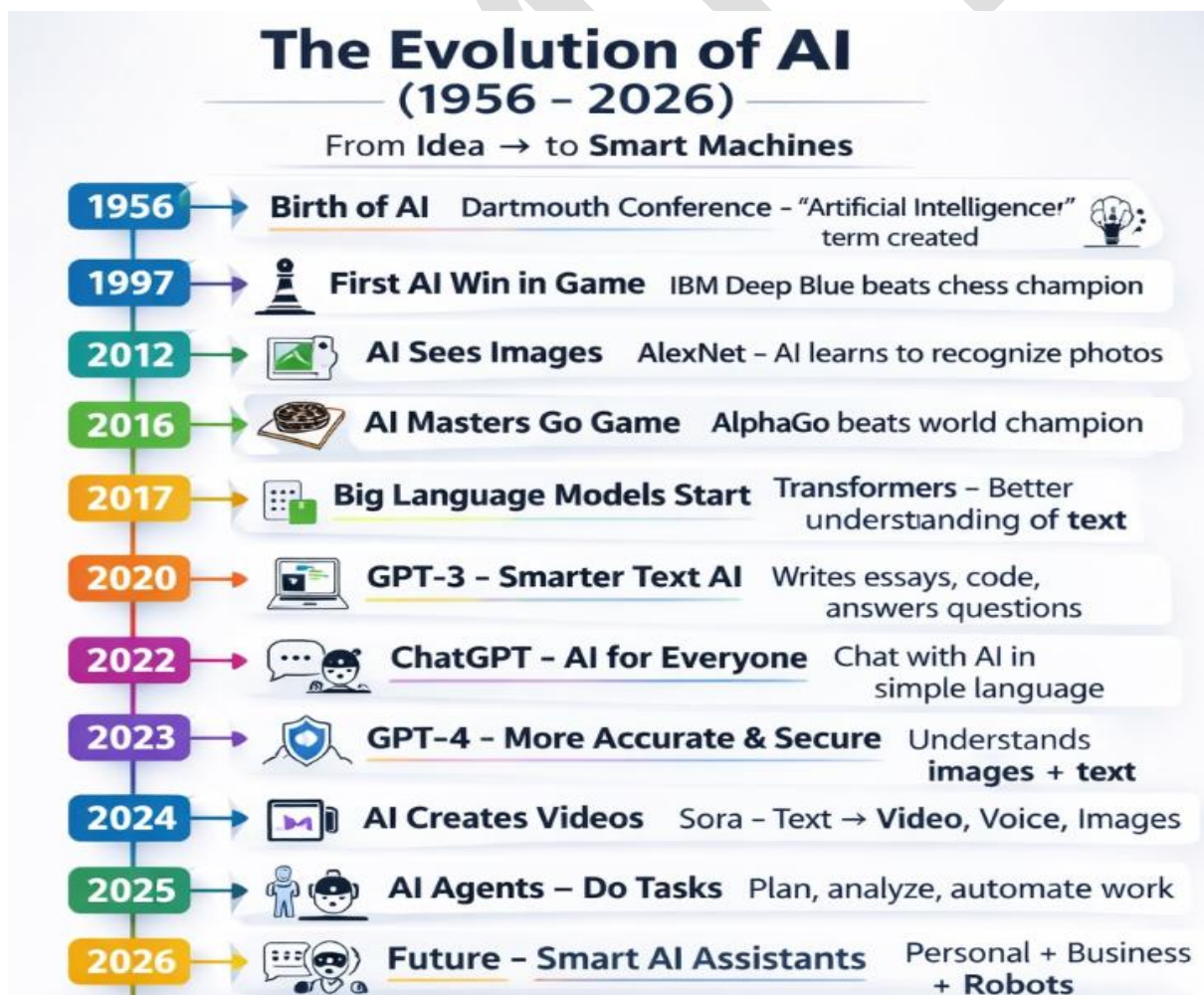
- Learning
- Reasoning
- Problem Solving
- Natural Language Processing
- Computer Vision
- Robotics

## 2. History of Artificial Intelligence

### Year    Development

|       |                                         |
|-------|-----------------------------------------|
| 1950  | Alan Turing proposed Turing Test        |
| 1956  | John McCarthy coined the term AI        |
| 1960  | Rule-Based Expert Systems               |
| 1980  | Expert Systems became popular           |
| 1997  | IBM Deep Blue defeated Garry Kasparov   |
| 2012  | Deep Learning revolution began          |
| 2016  | AlphaGo defeated Lee Sedol              |
| 2022+ | Generative AI (ChatGPT, Gemini, Claude) |

### Timeline



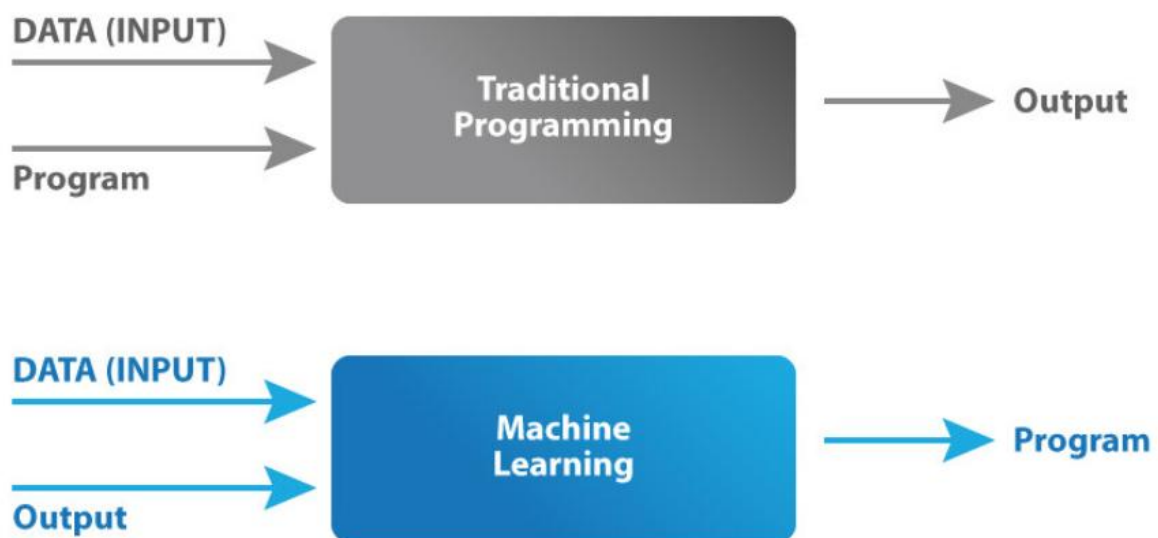
### 3. What is Machine Learning?

Machine Learning is a subset of Artificial Intelligence where computers learn patterns from data instead of being explicitly programmed.

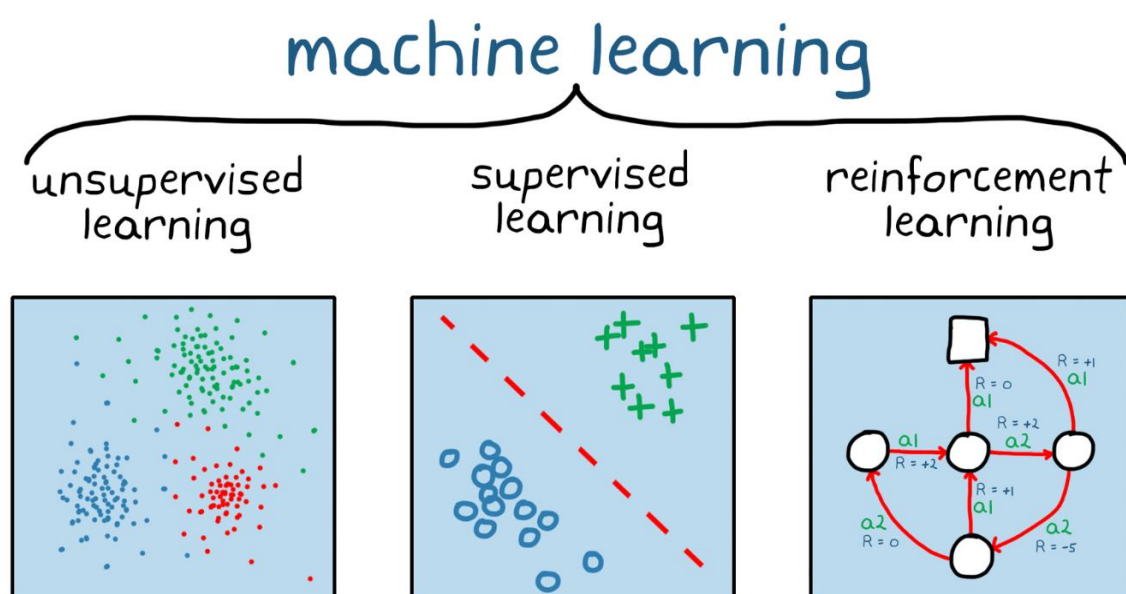
Instead of writing every rule manually, we provide examples (training data), and the model learns the relationship between inputs and outputs.

#### Definition

Machine Learning is the study of algorithms that improve automatically through experience using data.



#### Types of Machine Learning



## 1. Supervised Learning

Training data contains both inputs and correct outputs (labels).

Applications

- House price prediction
- Disease diagnosis
- Email spam detection

## 2. Unsupervised Learning

Only input data is provided. No labels are available.

Goal:

Find hidden patterns.

Applications

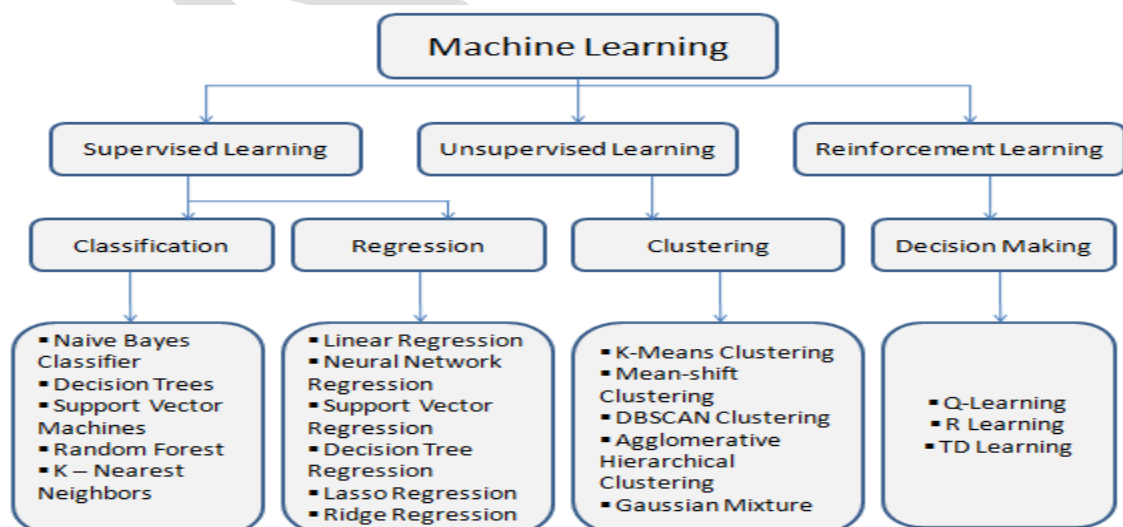
- Customer segmentation
- Market basket analysis

## 3. Reinforcement Learning

The agent learns through trial and error.

Applications

- Robotics
- Game Playing
- Autonomous Vehicles



## 4. What is Deep Learning?

Deep Learning is a specialized branch of Machine Learning that uses Artificial Neural Networks with multiple hidden layers to automatically learn complex patterns from large amounts of data.

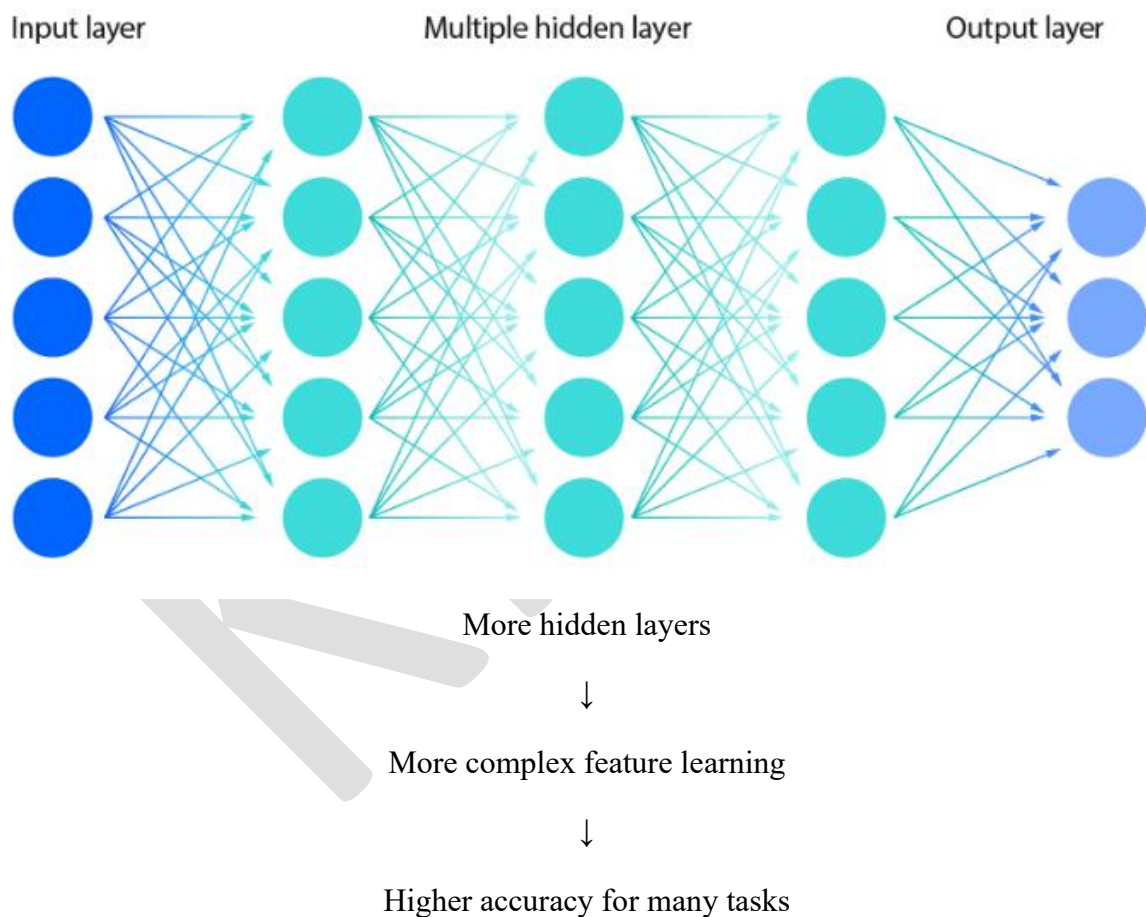
Instead of manually designing features, deep learning models learn useful features directly from raw data.

### Definition

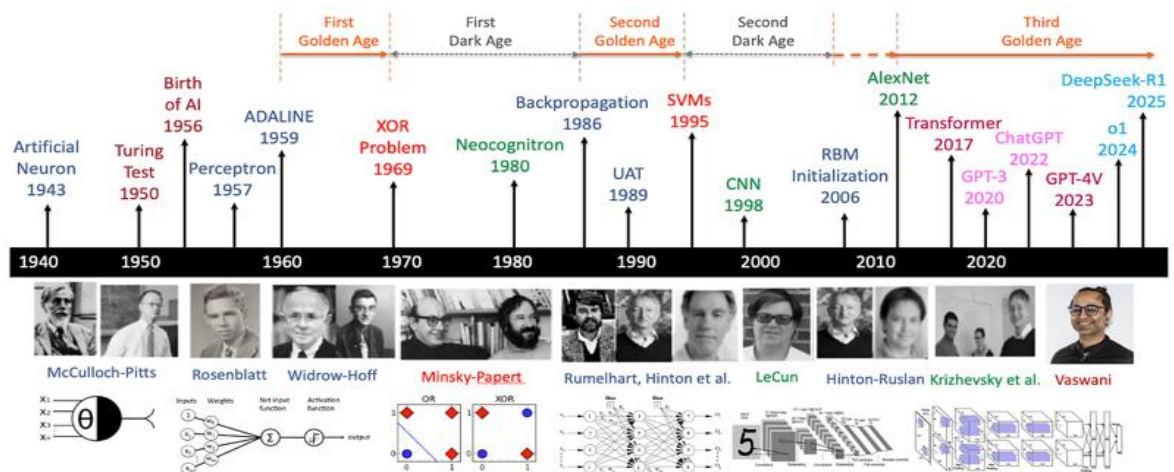
Deep Learning is a Machine Learning technique based on Artificial Neural Networks having multiple hidden layers.

### Why is it called "Deep"?

The word Deep refers to multiple hidden layers.



## 5. Evolution from AI to Deep Learning



## 6. AI vs Machine Learning vs Deep Learning

| Feature             | AI                               | ML               | DL                                |
|---------------------|----------------------------------|------------------|-----------------------------------|
| Definition          | Simulation of human intelligence | Learns from data | Learns using deep neural networks |
| Needs Data          | Sometimes                        | Yes              | Large amount                      |
| Programming         | Rule Based                       | Learns Rules     | Learns Features + Rules           |
| Accuracy            | Medium                           | High             | Very High                         |
| Hardware            | CPU                              | CPU/GPU          | GPU/TPU                           |
| Feature Engineering | Manual                           | Manual           | Automatic                         |

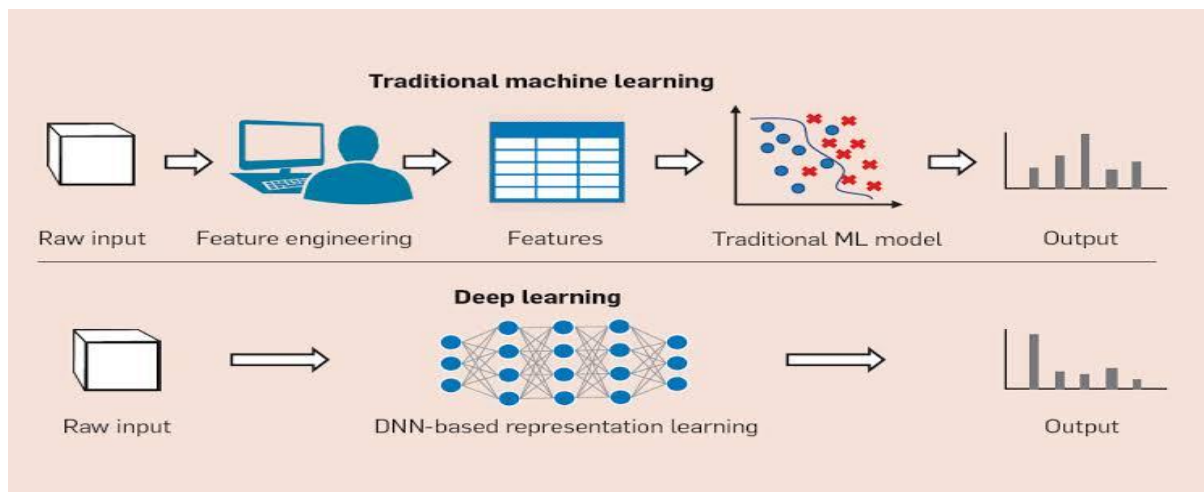
## 7. Why Deep Learning?

Traditional Machine Learning struggles with:

- Huge datasets
- Images
- Speech
- Videos
- Natural language

Deep Learning overcomes these by learning hierarchical representations automatically.

## Traditional Machine Learning/Deep Learning Pipeline



| Dimension                                                                           |                     | Machine Learning (ML)                            | Deep Learning (DL)                                             |
|-------------------------------------------------------------------------------------|---------------------|--------------------------------------------------|----------------------------------------------------------------|
|    | Data Volume         | Small to medium (hundreds-thousands of examples) | Large (millions of examples)                                   |
|    | Feature Engineering | Manual — you define the inputs                   | Automatic — model learns relevant features                     |
|   | Training Time       | Faster — minutes to hours                        | Slower — hours to weeks                                        |
|  | Hardware            | Runs on a standard CPU                           | Requires GPU / TPU (massive parallel processing)               |
|  | Interpretability    | More interpretable — clear feature importance    | Less interpretable — often a "black box"                       |
|  | Accuracy Ceiling    | Moderate — scope limited by feature quality      | High — can exceed human-level in specific applications         |
|  | Cost                | Lower — less compute resources, cheaper models   | Higher — expensive compute resources, costly models            |
|  | Failure Modes       | Fails gracefully — e.g., slightly underperforms  | Fails unpredictably — may produce noisy or nonsensical results |

## 8. Advantages of Deep Learning

### 1. Automatic Feature Extraction

No manual feature engineering.

### 2. High Accuracy

Especially for images, speech, and language tasks.

### 3. Learns Complex Relationships

Captures nonlinear patterns that simpler models cannot.

### 4. End-to-End Learning

Processes raw input directly to predictions.

### 5. Handles Large Datasets

Performance generally improves with more data.

## 9. Limitations of Deep Learning

### Large Dataset Requirement

Needs a large amount of labeled data.

### Computational Cost

Requires GPUs or TPUs for efficient training.

### Long Training Time

Training deep networks can take hours or days.

### Black Box Nature

Difficult to interpret why a particular decision was made.

### Overfitting

May memorize the training data if not properly regularized.

## 10. Applications of Deep Learning

### Computer Vision

- Face Recognition
- Object Detection
- Image Classification
- Medical Imaging

### Natural Language Processing (NLP)

- Language Translation
- Text Summarization
- Sentiment Analysis
- Chatbots

### Speech Recognition

- Voice Assistants
- Speech-to-Text
- Speaker Identification

### Autonomous Vehicles

Inputs:

- Camera
- LiDAR
- Radar
- GPS

↓

Deep Learning

↓

- Lane Detection
- Traffic Sign Recognition
- Obstacle Detection

### Healthcare

- Disease Prediction
- Cancer Detection
- MRI Analysis
- Drug Discovery

### Finance

- Fraud Detection
- Credit Scoring
- Stock Prediction (with caution)

### Robotics

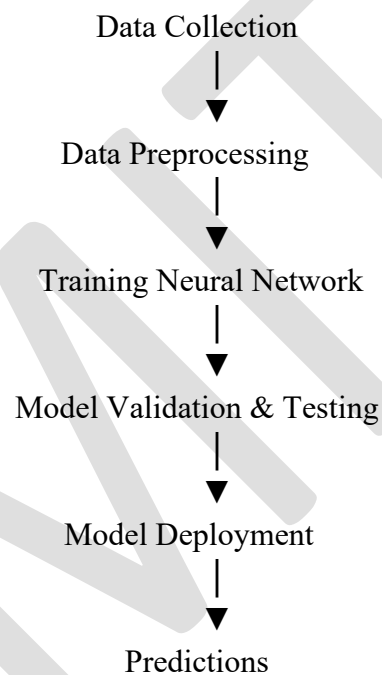
- Navigation
- Manipulation
- Industrial Automation

## 11. Real-World Deep Learning Examples

| Application        | Example               |
|--------------------|-----------------------|
| Face Recognition   | Unlocking smartphones |
| Speech Recognition | Voice assistants      |

| Application            | Example                            |
|------------------------|------------------------------------|
| Recommendation Systems | Movie and shopping recommendations |
| Medical Imaging        | Tumor detection                    |
| Self-Driving Cars      | Object and lane detection          |
| Language Translation   | Automatic translation systems      |

## 12. Deep Learning Workflow



### Key Points to Remember

- AI is the broad field of creating intelligent machines.
- Machine Learning is a subset of AI that learns from data.
- Deep Learning is a subset of Machine Learning based on multi-layer neural networks.
- Deep Learning performs automatic feature extraction.
- It excels in image, speech, and language tasks.
- Large datasets and powerful hardware significantly improve deep learning performance.

## **Viva Questions**

1. What is Artificial Intelligence?
2. What is Machine Learning?
3. What is Deep Learning?
4. Differentiate AI, ML, and DL.
5. What are the advantages of Deep Learning?
6. What are its limitations?
7. List five applications of Deep Learning.
8. Why are GPUs commonly used for Deep Learning?
9. What is automatic feature extraction?

# Unit 1: INTRODUCTION TO DEEP LEARNING

## SESSION2:

### Fundamentals of Neural Networks

#### Learning Objectives

- Understand biological neurons and artificial neurons.
- Explain the McCulloch–Pitts neuron model.
- Understand the Perceptron learning model.
- Describe Single-Layer and Multi-Layer Perceptrons.
- Calculate neuron outputs using weights and bias.
- Understand forward propagation.
- Interpret neural network architectures.

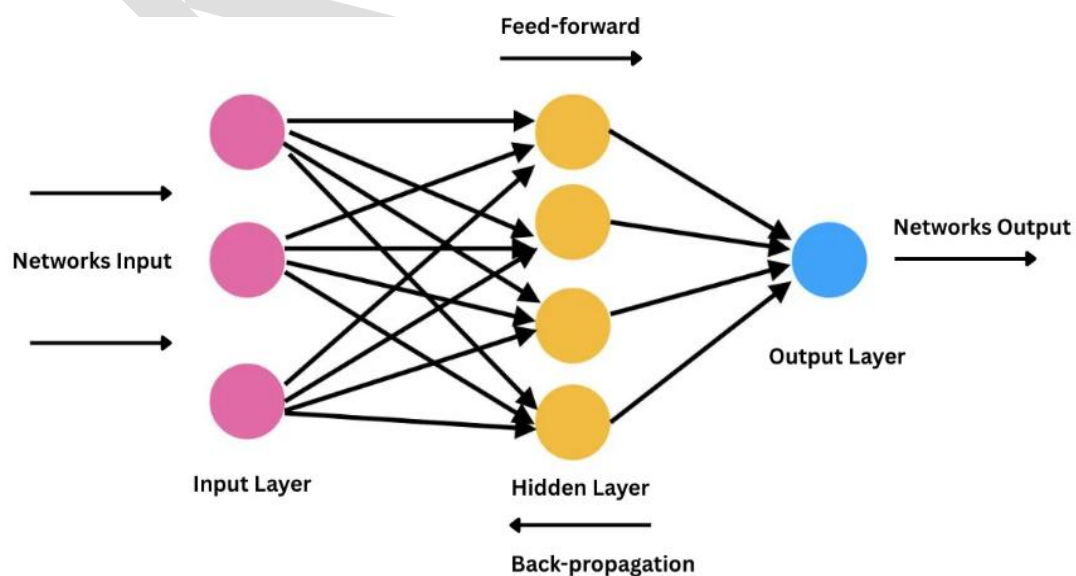
#### 1. Introduction to Neural Networks

A Neural Network is a computational model inspired by the human brain. It consists of interconnected processing units called neurons, which work together to learn patterns from data.

Instead of following predefined rules, neural networks learn from examples by adjusting internal parameters called weights.

#### Definition

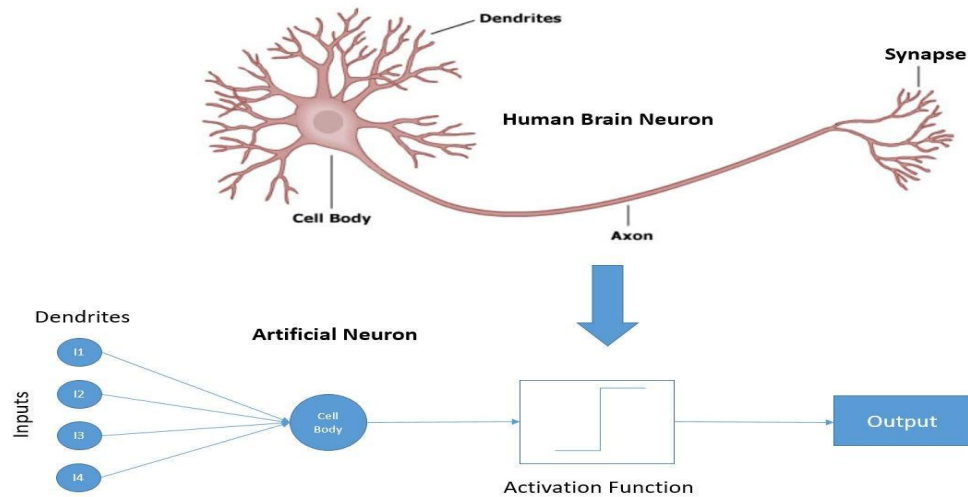
An Artificial Neural Network (ANN) is a network of interconnected artificial neurons that processes information and learns from training data to make predictions or decisions.



## 2. Biological Neuron

The human brain contains approximately 86 billion neurons. These neurons communicate through electrical and chemical signals.

### Structure of a Biological Neuron



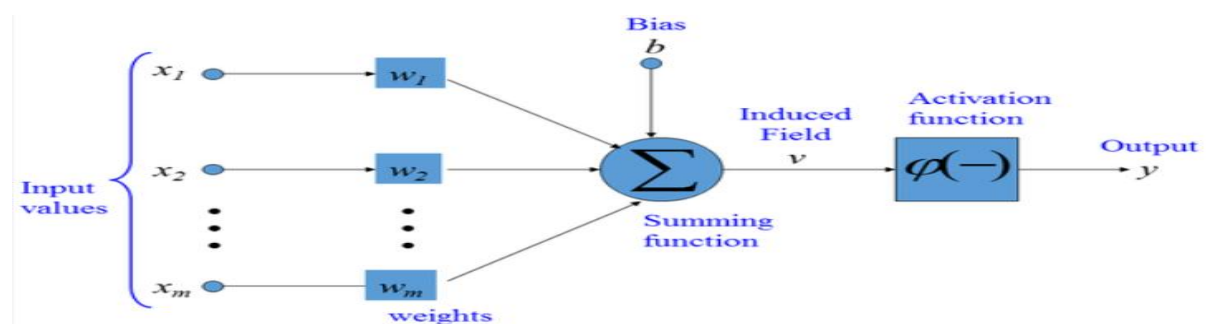
### Functions of Each Part

| Part             | Function                             |
|------------------|--------------------------------------|
| Dendrites        | Receive signals from other neurons   |
| Cell Body (Soma) | Processes the received signals       |
| Axon             | Transmits the signal                 |
| Axon Terminals   | Passes the signal to the next neuron |

## 3. Artificial Neuron

Artificial neurons mimic the working of biological neurons using mathematical operations.

### Structure



Where:

- $x_1, x_2, x_3$  = Inputs
- $w_1, w_2, w_3$  = Weights
- $b$  = Bias
- $\Sigma$  = Summation
- $f()$  = Activation Function
- $y$  = Output

#### 4. Components of an Artificial Neuron

##### (a) Input (x)

Inputs are the features provided to the model.

Example:

- Height
- Weight
- Age

These become:

$x_1, x_2, x_3$

##### (b) Weight (w)

Weights determine the importance of each input.

- Large positive weight  $\rightarrow$  strong influence
- Negative weight  $\rightarrow$  opposite influence
- Small weight  $\rightarrow$  less influence

##### (c) Bias (b)

Bias shifts the activation function and helps the neuron learn more effectively.

Without bias, the model becomes less flexible.

##### (d) Summation

The neuron computes the weighted sum:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

This is called the net input.

### (e) Activation Function

The activation function introduces non-linearity.

Without it, even deep networks behave like a simple linear model.

$$y = f(z)$$

## 5. Mathematical Model of a Neuron

For  $n$  inputs:

$$z = \sum_{i=1}^n w_i x_i + b$$

Output:

$$y = f(z)$$

This is the fundamental equation of every artificial neuron.

## 6. Numerical Example

Given:

$$x_1 = 2, x_2 = 1, x_3 = 3$$

Weights:

$$w_1 = 0.5, w_2 = 0.2, w_3 = 0.4$$

Bias:

$$b = 0.3$$

### Step 1: Weighted Sum

$$\begin{aligned} z &= (2)(0.5) + (1)(0.2) + (3)(0.4) + 0.3 \\ &= 1 + 0.2 + 1.2 + 0.3 \\ z &= 2.7 \end{aligned}$$

If no activation function is applied:

$$y = 2.7$$

## 7. McCulloch–Pitts Neuron Model

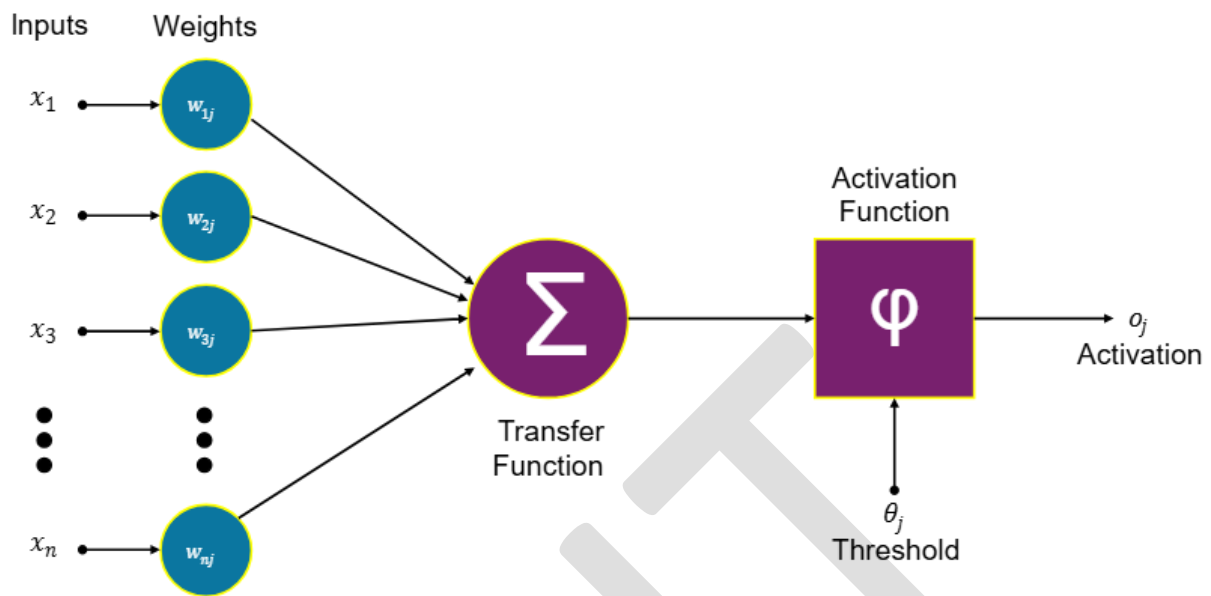
The McCulloch–Pitts (MCP) neuron, proposed in 1943, is the first mathematical model of an artificial neuron.

It:

- Uses binary inputs (0 or 1)
- Produces binary outputs (0 or 1)

- Compares the weighted sum against a threshold

### Diagram



### Decision Rule

If

$$\sum w_i x_i \geq \theta$$

Output = 1

Otherwise

Output = 0

where  **$\theta$  (theta)** is the threshold.

### Example

Inputs:

$$x_1 = 1, x_2 = 0, x_3 = 1$$

Weights:

$$1, 1, 1$$

Threshold:

$$\theta = 2$$

Weighted sum:

$$1 + 0 + 1 = 2$$

Since

$$2 \geq 2$$

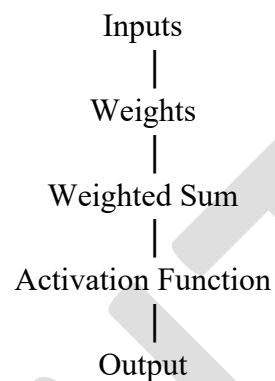
Output = 1

## 8. Perceptron

The Perceptron, introduced by Frank Rosenblatt (1958), is the first trainable neural network model.

Unlike the MCP neuron, the perceptron can learn by adjusting its weights during training.

### Structure



### Output Equation

$$y = f(\sum w_i x_i + b)$$

## 9. Single-Layer Perceptron (SLP)

A Single-Layer Perceptron has:

- One input layer
- One output layer
- No hidden layers

Input Layer ----- Output Layer

### Advantages

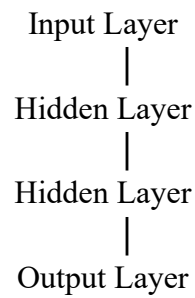
- Simple
- Fast
- Easy to train

### Limitations

Cannot solve non-linear problems such as the XOR problem.

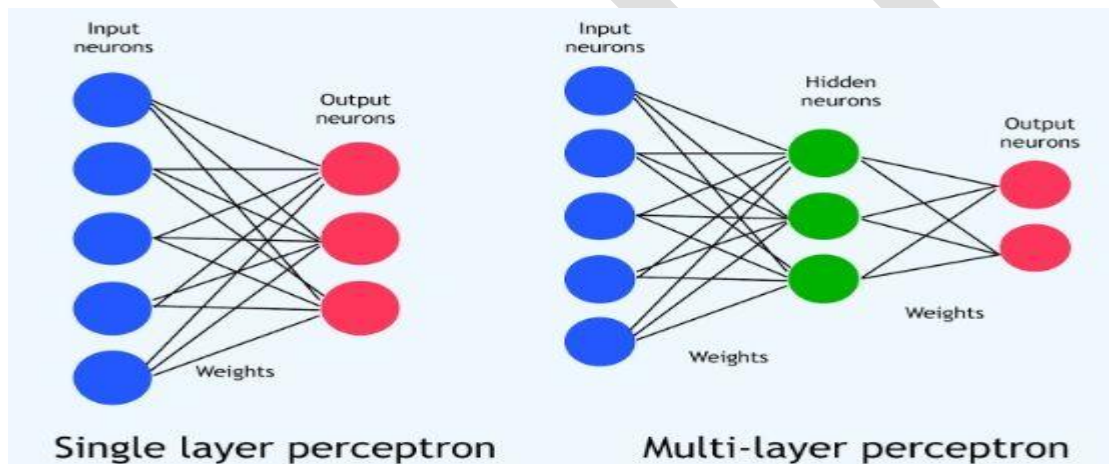
## 10. Multi-Layer Perceptron (MLP)

An MLP contains one or more hidden layers.



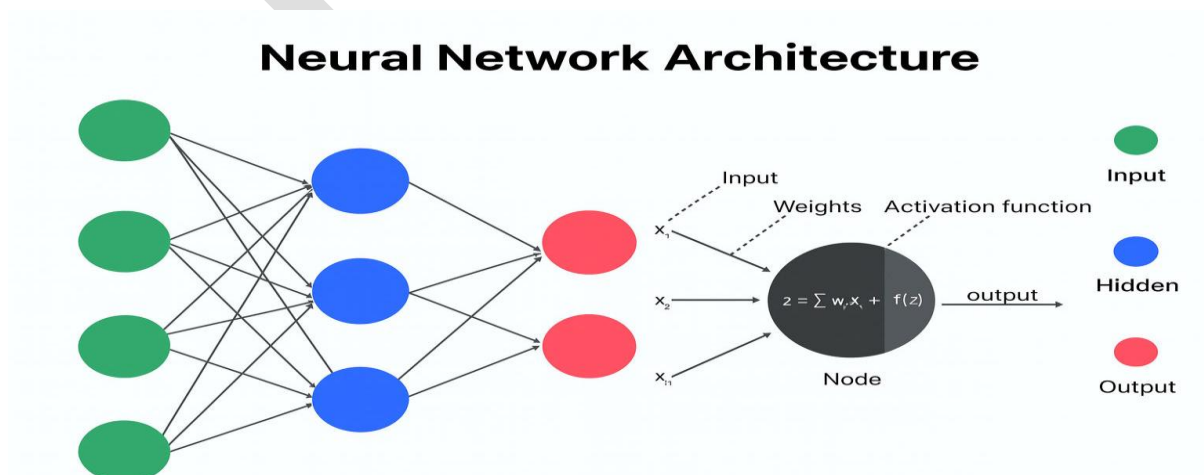
Advantages:

- Learns complex patterns
- Handles non-linear data
- Foundation of Deep Learning



## 11. Neural Network Architecture

A typical feedforward neural network:



## 12. Layers in a Neural Network

### Input Layer

Receives input features.

### Hidden Layer

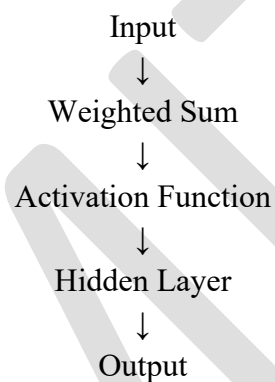
Performs feature extraction and learning.

### Output Layer

Produces the final prediction.

## 13. Forward Propagation

Forward propagation is the process of passing data from the input layer to the output layer.



Each neuron computes:

$$z = w^T x + b$$

Then:

$$a = f(z)$$

The output of one layer becomes the input to the next.

## 14. Why Hidden Layers?

Hidden layers enable the network to learn increasingly complex features.

Example for image recognition:

- Layer 1: Detects edges
- Layer 2: Detects corners and textures
- Layer 3: Detects object parts (eyes, wheels)
- Layer 4: Recognizes complete objects (face, car)

## 15. Advantages of Neural Networks

- Learns complex relationships

- Works well with large datasets
- High prediction accuracy
- Automatically extracts features
- Supports image, speech, and text applications

## 16. Limitations

- Requires large amounts of data
- Computationally intensive
- Long training time
- Difficult to interpret (black-box nature)
- Can overfit if not properly regularized

## Key Formula Summary

### 1. Weighted Sum

$$z = \sum_{i=1}^n w_i x_i + b$$

### Neuron Output

$$y = f(z)$$

### MCP Decision Rule

$$\text{Output} = \begin{cases} 1, & \sum w_i x_i \geq \theta \\ 0, & \text{otherwise} \end{cases}$$

## Key Points to Remember

- Neural networks are inspired by biological neurons.
- An artificial neuron consists of inputs, weights, bias, summation, and an activation function.
- The weighted sum is the core computation of every neuron.
- The Perceptron is the first trainable neural network.
- Multi-Layer Perceptrons (MLPs) are the basis of modern deep learning.
- Forward propagation passes information from the input layer to the output layer.

## Viva Questions

1. What is an Artificial Neural Network (ANN)?
2. Draw and explain the structure of a biological neuron.
3. Explain the components of an artificial neuron.
4. What is the role of weights in a neural network?
5. Why is bias required?
6. Explain the McCulloch–Pitts neuron model.
7. What is a Perceptron?
8. Differentiate between a Single-Layer Perceptron and a Multi-Layer Perceptron.
9. What is forward propagation?
10. Why are hidden layers important?

## SESSION 3

### Activation Functions in Deep Learning

#### Learning Objectives

- Define an activation function.
- Explain why activation functions are required.
- Understand different types of activation functions.
- Derive their mathematical equations.
- Select the appropriate activation function for different deep learning problems.

#### 1. What is an Activation Function?

An activation function is a mathematical function applied to the output of a neuron after computing the weighted sum.

The neuron first calculates:

$$z = \sum_{i=1}^n w_i x_i + b$$

Then applies the activation function:

$$y = f(z)$$

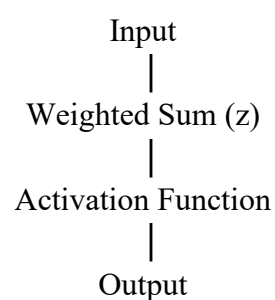
where:

- $z$  = weighted sum (net input)
- $f(z)$  = activation function
- $y$  = neuron output

#### 2. Why Do We Need Activation Functions?

Without an activation function, every neuron performs only a linear transformation. Even if many layers are stacked, the entire network behaves like a single linear model and cannot learn complex relationships.

#### With Activation Function



Activation functions introduce non-linearity, enabling neural networks to learn complex decision boundaries.

### 3. Types of Activation Functions

The commonly used activation functions are:

1. Binary Step Function
2. Linear Function
3. Sigmoid Function
4. Tanh Function
5. ReLU (Rectified Linear Unit)
6. Leaky ReLU
7. ELU (Exponential Linear Unit)
8. Softmax Function

#### 3.1 Binary Step Function

##### Definition

The Binary Step Function outputs **0 or 1** based on a threshold.

##### Formula

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

##### Example

If  $x = 5$ , then:

$$f(5) = 1$$

If  $x = -2$ , then:

$$f(-2) = 0$$

##### Advantages

- Very simple.
- Fast computation.

##### Disadvantages

- Not differentiable.
- Cannot be used for backpropagation.
- Suitable only for simple binary decisions.

## 3.2 Linear Activation Function

### Formula

$$f(x) = x$$

### Characteristics

- Output equals input.
- No non-linearity.

### Advantages

- Simple to compute.

### Disadvantages

- Deep networks reduce to a single linear transformation.
- Cannot solve complex problems.

## 3.3 Sigmoid Function

The Sigmoid Function is one of the earliest activation functions used in neural networks.

### Formula

$$f(x) = \frac{1}{1 + e^{-x}}$$

### Output Range

$$0 < f(x) < 1$$

### Example

If  $x = 0$ :

$$f(0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

### Advantages

- Smooth and differentiable.
- Outputs probabilities (0 to 1).
- Useful for binary classification output layers.

### Disadvantages

- Vanishing gradient problem.
- Slow convergence.
- Outputs are not zero-centered.

### 3.4 Tanh (Hyperbolic Tangent)

The Tanh function is an improved version of Sigmoid.

#### Formula

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

#### Output Range

$$-1 < f(x) < 1$$

#### Example

If  $x = 0$ :

$$f(0) = 0$$

#### Advantages

- Zero-centered output.
- Learns faster than Sigmoid.
- Better gradient flow.

#### Disadvantages

- Still suffers from vanishing gradients for large positive or negative inputs.

### 3.5 ReLU (Rectified Linear Unit)

ReLU is the most widely used activation function in modern deep learning.

#### Formula

$$f(x) = \max(0, x)$$

or

$$f(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

#### Example

If  $x = 5$ :

$$f(5) = 5$$

If  $x = -4$ :

$$f(-4) = 0$$

#### Advantages

- Very fast.

- Avoids vanishing gradients for positive values.
- Simple implementation.
- Accelerates training.

### Disadvantages

- Dying ReLU problem: neurons receiving only negative inputs may stop learning.

### 3.6 Leaky ReLU

Leaky ReLU addresses the dying ReLU problem by allowing a small, non-zero output for negative inputs.

#### Formula

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$

#### Advantages

- Prevents dead neurons.
- Faster convergence than Sigmoid and Tanh.
- Better gradient propagation.

#### Disadvantages

- Requires choosing the leakage coefficient (e.g., 0.01).

### 3.7 ELU (Exponential Linear Unit)

#### Formula

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

where  $\alpha$  is typically 1.

#### Advantages

- Smooth output for negative values.
- Reduces bias shift.
- Faster learning than ReLU in some cases.

#### Disadvantages

- More computationally expensive than ReLU.

### 3.8 Softmax Function

Softmax is used for multi-class classification.

## Formula

For class  $i$ :

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

## Example

Suppose a model predicts:

- Cat = 2.5
- Dog = 1.0
- Bird = 0.5

Softmax converts these scores into probabilities such as:

- Cat = 0.72
- Dog = 0.18
- Bird = 0.10

The probabilities sum to **1**.

## Advantages

- Produces a valid probability distribution.
- Ideal for multi-class classification.

## Disadvantages

- Used only in the output layer for classification.

#### 4. Comparison of Activation Functions

| Activation  | Formula                   | Output Range        | Advantages               | Disadvantages                    | Applications                   |
|-------------|---------------------------|---------------------|--------------------------|----------------------------------|--------------------------------|
| Binary Step | 0 or 1                    | {0,1}               | Simple                   | Not differentiable               | Logic gates                    |
| Linear      | $f(x) = x$                | $-\infty, \infty$   | Easy                     | Cannot learn non-linear patterns | Regression output              |
| Sigmoid     | $\frac{1}{1 + e^{-x}}$    | (0,1)               | Probability output       | Vanishing gradient               | Binary classification          |
| Tanh        | Hyperbolic tangent        | (-1,1)              | Zero-centered            | Vanishing gradient               | Hidden layers (older networks) |
| ReLU        | $\max(0, x)$              | $[0, \infty)$       | Fast, efficient          | Dying ReLU                       | Hidden layers (most common)    |
| Leaky ReLU  | $x$ or $0.01x$            | $(-\infty, \infty)$ | Prevents dead neurons    | Requires parameter               | Hidden layers                  |
| ELU         | Exponential               | $(-\alpha, \infty)$ | Smooth learning          | Computationally expensive        | Deep networks                  |
| Softmax     | Exponential normalization | (0,1)               | Probability distribution | Output layer only                | Multi-class classification     |

#### Practical Guidelines

- **Regression problems:** Linear activation (output layer).
- **Binary classification:** Sigmoid (output layer).
- **Multi-class classification:** Softmax (output layer).
- **Hidden layers in modern deep networks:** ReLU (default choice).
- **If neurons become inactive:** Leaky ReLU or ELU.

| Activation Function | Formula                                                                            | Common Applications                        | Advantages                                                             | Disadvantages                                                      |
|---------------------|------------------------------------------------------------------------------------|--------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------|
| Linear              | $f(x) = x$                                                                         | Rarely used in modern neural networks      | None                                                                   | Does not introduce non-linearity                                   |
| Binary              | $f(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$ | Classification tasks with binary output    | Simple and efficient                                                   | Limited to binary classification tasks                             |
| Sigmoid             | $f(x) = \frac{1}{1 + e^{-x}}$                                                      | Classification tasks                       | Output can be interpreted as probability                               | Saturates for large positive or negative inputs; not zero-centered |
| Tanh                | $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$                                         | Classification tasks                       | Zero-centered; does not saturate for large positive or negative inputs | None                                                               |
| ReLU                | $f(x) = \max(0, x)$                                                                | Widely used in modern neural networks      | Simple and efficient; does not saturate for large inputs               | Can suffer from "dying ReLU" problem; not zero-centered            |
| Softmax             | $f(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$                                          | Classification tasks with multiple classes | Outputs probability distribution over classes                          | None                                                               |

### Key Points to Remember

- Activation functions introduce non-linearity, enabling neural networks to solve complex problems.
- ReLU is the most commonly used activation function for hidden layers.
- Sigmoid is mainly used for binary classification output.
- Softmax is used for multi-class classification.
- Leaky ReLU and ELU improve upon standard ReLU by addressing its limitations.

### Viva Questions

1. What is an activation function?
2. Why is an activation function necessary in deep learning?
3. Explain the Sigmoid function with its equation.
4. What is the ReLU function? Why is it popular?
5. What is the Dying ReLU problem?
6. Explain Leaky ReLU.

## SESSION 4

# Loss Functions, Gradient Descent, Optimization Methods, and Backpropagation

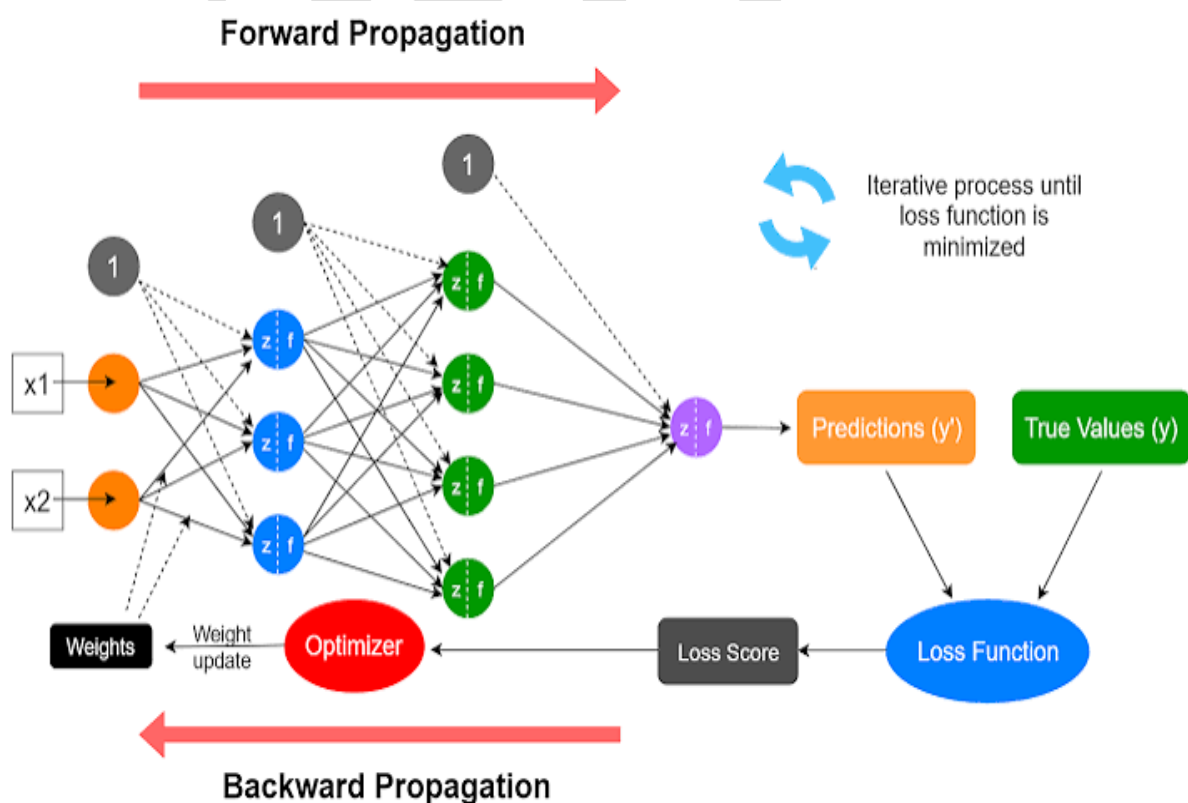
### Learning Objectives

- Understand the concept of a loss (cost) function.
- Differentiate between loss and cost.
- Explain Mean Squared Error (MSE) and Cross-Entropy Loss.
- Understand optimization and Gradient Descent.
- Explain Batch, Stochastic, and Mini-Batch Gradient Descent.
- Understand the concept of learning rate.
- Explain Backpropagation with mathematical formulas and a numerical example.

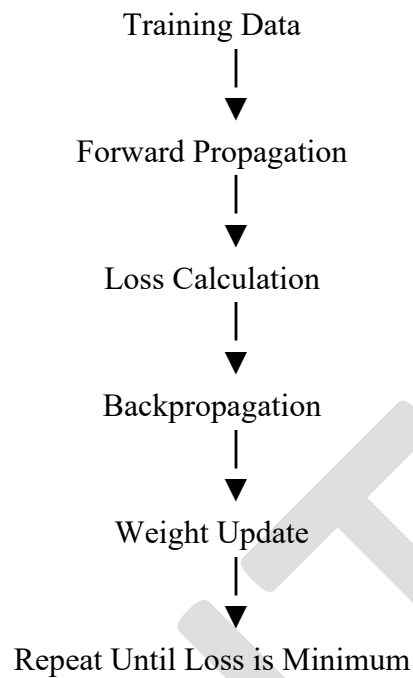
### 1. Introduction

A neural network initially makes random predictions because its weights and biases are randomly initialized.

The goal of training is to adjust these weights and biases so that the predictions become as close as possible to the actual target values.



This learning process consists of four major steps:



## 2. What is a Loss Function?

A Loss Function measures the error for a single training example by comparing the predicted output with the actual output.

### Formula

$$\text{Loss} = f(\text{Actual}, \text{Predicted})$$

If the prediction is correct, the loss is small. If the prediction is incorrect, the loss is large.

### Example

Actual House Price = ₹50 Lakhs

Predicted House Price = ₹48 Lakhs

Error = ₹2 Lakhs

The loss function quantifies this error.

## 3. What is a Cost Function?

The Cost Function is the average loss over all training examples.

Suppose there are  $N$  training samples.

$$J = \frac{1}{N} \sum_{i=1}^N L_i$$

Where:

- $J$  = Cost Function
- $L_i$  = Loss for the  $i^{th}$  training example
- $N$  = Number of training samples

### Difference Between Loss and Cost

| Loss Function                     | Cost Function                     |
|-----------------------------------|-----------------------------------|
| Error for one sample              | Average error over all samples    |
| Used during individual prediction | Used to evaluate the entire model |

## 4. Mean Squared Error (MSE)

MSE is the most common loss function for regression problems.

### Formula

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Where:

- $y_i$  = Actual value
- $\hat{y}_i$  = Predicted value
- $N$  = Number of samples

### Numerical Example

Actual values:

[5, 7, 9]

Predicted values:

[4, 8, 10]

Errors:

$$(5 - 4)^2 = 1$$

$$(7 - 8)^2 = 1$$

$$(9 - 10)^2 = 1$$

$$MSE = \frac{1 + 1 + 1}{3} = 1$$

## 5. Cross-Entropy Loss

Cross-Entropy Loss is used for classification problems.

For binary classification:

$$L = -[y\log(\hat{y}) + (1 - y)\log(1 - \hat{y})]$$

Where:

- $y$  = Actual label (0 or 1)
- $\hat{y}$  = Predicted probability

### Example

Actual = 1

Predicted Probability = 0.9

Loss:

$$L = -(1 \times \log(0.9)) \approx 0.105$$

A prediction close to the actual label results in a small loss.

## 6. What is Optimization?

Optimization is the process of finding the best values of weights and biases that minimize the cost function.

The optimization process repeatedly updates the parameters to reduce the error.

## 7. Gradient Descent

Gradient Descent is the most widely used optimization algorithm in deep learning.

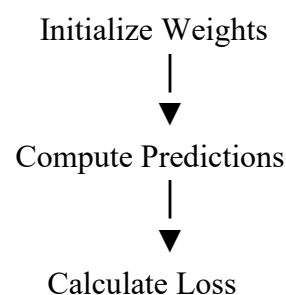
### Objective

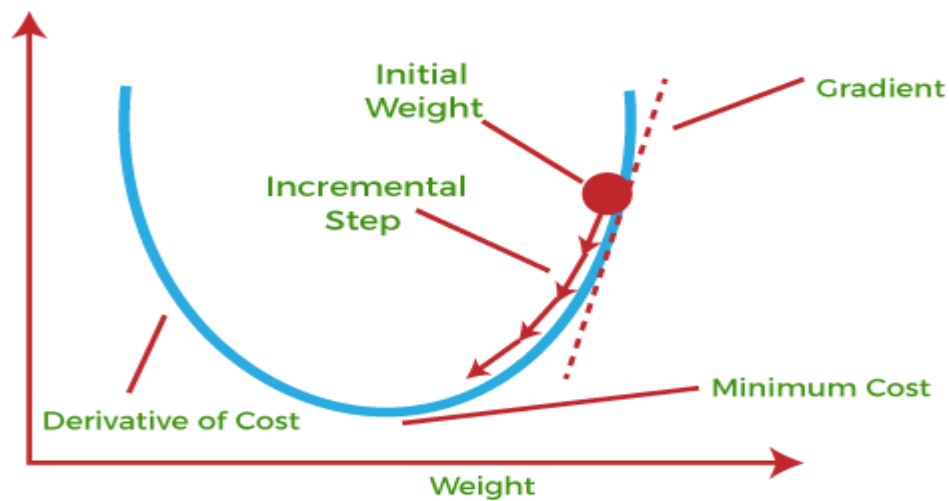
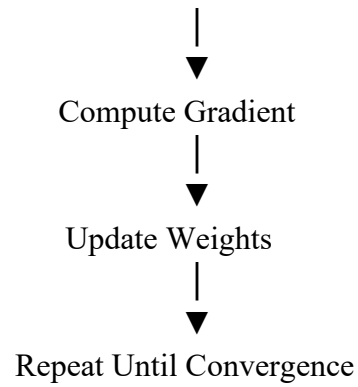
Find the values of weights that minimize the cost function.

### Basic Idea

Imagine standing on top of a mountain and trying to reach the lowest point (the minimum cost). At each step, you move downhill in the direction of the steepest descent.

### Gradient Descent Process





## 8. Learning Rate

The Learning Rate ( $\eta$ ) controls the size of each weight update.

### Weight Update Formula

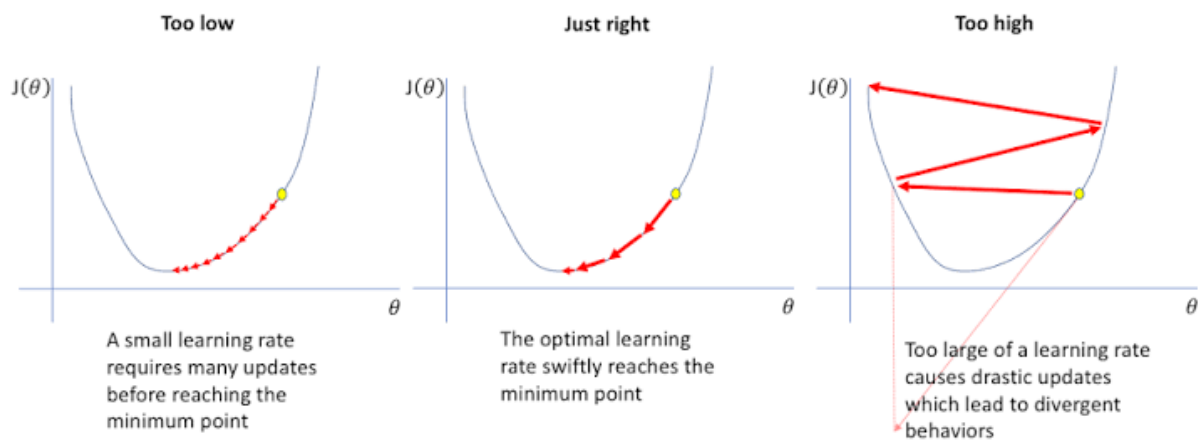
$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial J}{\partial w}$$

Where:

- $w$  = Weight
- $J$  = Cost Function
- $\eta$  = Learning Rate
- $\frac{\partial J}{\partial w}$  = Gradient

### Effect of Learning Rate

- **Too Small:** Training is very slow.
- **Too Large:** Model may overshoot the minimum and fail to converge.
- **Optimal:** Fast and stable convergence.



## 9. Types of Gradient Descent

### (a) Batch Gradient Descent

Uses the entire training dataset to compute the gradient before updating weights.

#### Advantages

- Stable updates
- Smooth convergence

#### Disadvantages

- Slow for large datasets
- High memory usage

### b) Stochastic Gradient Descent (SGD)

Updates weights after each training sample.

#### Advantages

- Faster updates
- Escapes local minima more easily

#### Disadvantages

- Noisy convergence
- Fluctuating loss

### (c) Mini-Batch Gradient Descent

Uses a small batch of samples (e.g., 32, 64, 128) before updating weights.

#### Advantages

- Faster than Batch GD

- More stable than SGD
- Efficient GPU utilization

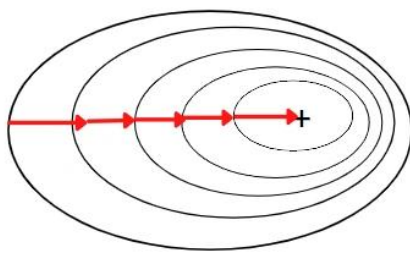
### Why It Is Preferred

Most modern deep learning frameworks (TensorFlow, PyTorch) use Mini-Batch Gradient Descent because it balances speed and stability.

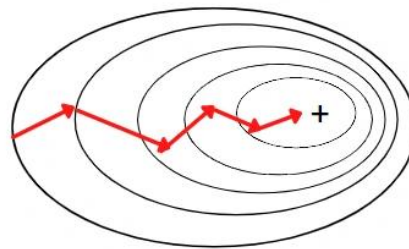
### 10. Comparison of Gradient Descent Methods

| Method     | Data Used per Update | Speed | Stability |
|------------|----------------------|-------|-----------|
| Batch      | Entire dataset       | Slow  | High      |
| Stochastic | One sample           | Fast  | Low       |
| Mini-Batch | Small batch          | Fast  | High      |

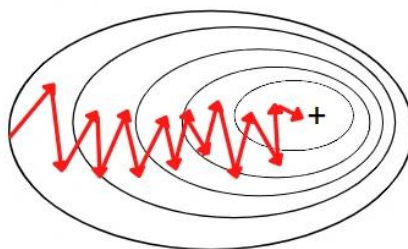
**Batch Gradient Descent**



**Mini-Batch Gradient Descent**



**Stochastic Gradient Descent**



### 11. Backpropagation

Backpropagation is the algorithm used to train neural networks by updating weights based on the error.

#### Why Is It Needed?

If the prediction is wrong, the network must determine:

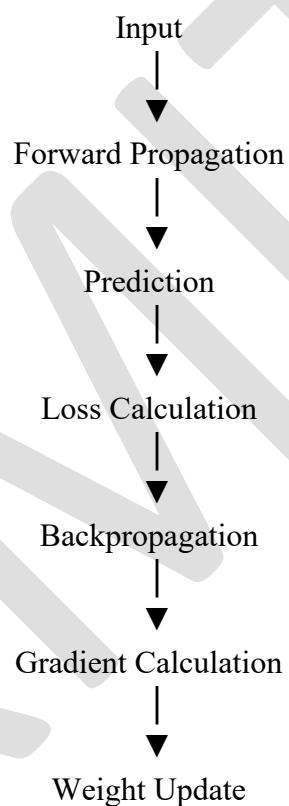
- Which weights caused the error?
- By how much should each weight be adjusted?

Backpropagation answers these questions by computing gradients and propagating the error backward through the network.

## 12. Steps in Backpropagation

1. Perform Forward Propagation.
2. Compute the Loss.
3. Calculate the Gradient of the loss with respect to each weight.
4. Update the weights using Gradient Descent.
5. Repeat until the loss is minimized.

### Backpropagation Flow



## 13. Numerical Example of Weight Update

Suppose:

Current weight:

$$w = 0.50$$

Learning rate:

$$\eta = 0.10$$

Gradient:

$$\frac{\partial J}{\partial w} = 0.20$$

Using the update rule:

$$w_{\text{new}} = 0.50 - (0.10 \times 0.20)$$

$$w_{\text{new}} = 0.50 - 0.02 = 0.48$$

The weight decreases from **0.50 to 0.48**, reducing the model's error.

### SOLVED EXAMPLE:

Suppose we have a neuron with:

- Current Weight:  $w = 0.50$
- Learning Rate:  $\eta = 0.10$
- Gradient:  $\frac{\partial J}{\partial w} = 0.20$

We use the **Gradient Descent Weight Update Rule**:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial J}{\partial w}$$

where:

- $w_{\text{new}}$ : Updated weight
- $w_{\text{old}}$ : Current weight
- $\eta$ : Learning rate
- $\frac{\partial J}{\partial w}$ : Gradient (slope of the cost function)

### Step 1: Initial Weight

Assume the weight connecting one neuron to another is:

$$w = 0.50$$

This is the value currently used by the neural network.

Neuron A -----( Weight = 0.50 )-----> Neuron B

Initially, weights are often assigned **random values**.

### Step 2: Forward Propagation

Suppose:

- Input:

$$x = 2$$

Current weight:

$$w = 0.50$$

Bias:

$$b = 0$$

The neuron computes the weighted sum:

$$z = wx + b$$

Substituting the values:

$$z = (0.50)(2) + 0$$

$$z = 1$$

If we use a **linear activation function**, the output is:

$$y = 1$$

### Step 3: Compare with Actual Output

Suppose:

Actual output:

$$y = 2$$

Predicted output:

$$\hat{y} = 1$$

Since:

$$1 \neq 2$$

there is an error, so the model needs to learn by adjusting its weight.

### Step 4: Calculate the Error

A simple error measure is:

$$\text{Error} = \text{Actual} - \text{Predicted}$$

$$= 2 - 1$$

$$= 1$$

The prediction is lower than the actual value.

### Step 5: Calculate the Cost (Loss)

For one sample, using Mean Squared Error:

$$J = \frac{1}{2}(y - \hat{y})^2$$

Substitute:

$$J = \frac{1}{2}(2 - 1)^2$$

$$= \frac{1}{2}(1)^2$$

$$= 0.5$$

The current cost is **0.5**.

Our goal is to reduce this value as much as possible.

### Step 6: Compute the Gradient

The **gradient** tells us:

- In which direction should the weight move?
- How much should the weight change?

Suppose backpropagation calculates:

$$\frac{\partial J}{\partial w} = 0.20$$

This means:

- The cost increases as the weight increases.
- Therefore, the weight should be decreased.

### Step 7: Choose the Learning Rate

Suppose:

$$\eta = 0.10$$

The learning rate determines how large the update step will be.

- Small learning rate → slow learning.
- Large learning rate → may overshoot the minimum.
- Moderate learning rate → stable learning.

### Step 8: Update the Weight

Apply the Gradient Descent formula:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial J}{\partial w}$$

Substitute the values:

$$w_{\text{new}} = 0.50 - (0.10)(0.20)$$

Multiply:

$$0.10 \times 0.20 = 0.02$$

So,

$$w_{\text{new}} = 0.50 - 0.02$$

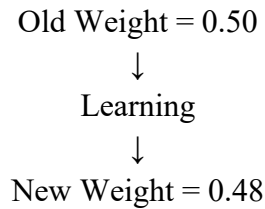
$$w_{\text{new}} = 0.48$$

The updated weight is:

**0.48**

### Step 9: What Does This Mean?

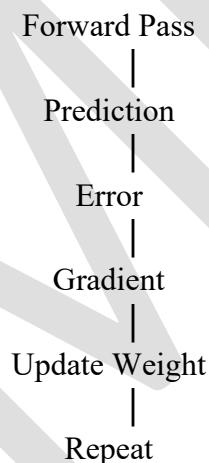
The weight has changed:



This small adjustment helps the model produce a prediction that is closer to the correct answer in the next iteration.

### Step 10: Repeat the Process

The neural network repeats this process many times.



Each repetition is called an iteration.

A complete pass through the entire training dataset is called an epoch.

### Step 11: Effect of Learning Rate

#### Case 1: Small Learning Rate

$$\eta = 0.001$$

Weight update:

$$0.50 - (0.001 \times 0.20) = 0.4998$$

The weight changes very little, so learning is slow.

#### Case 2: Medium Learning Rate

$$\eta = 0.10$$

Weight update:

$$0.48$$

This is a balanced update and often leads to stable learning.

### Case 3: Large Learning Rate

$$\eta = 5$$

Weight update:

$$0.50 - (5 \times 0.20) = 0.50 - 1 = -0.50$$

The weight changes too much and may **overshoot** the optimal value, causing unstable training.

### Step 12: Intuition Behind Gradient Descent

Imagine you are standing on a mountain and want to reach the lowest point.

- If the slope goes upward, move in the opposite direction.
- If the slope goes downward, continue moving downhill.

The gradient tells you the direction of the steepest increase, so Gradient Descent moves in the opposite direction to minimize the cost.

### Step 13: Why Is the Gradient Subtracted?

The update rule is:

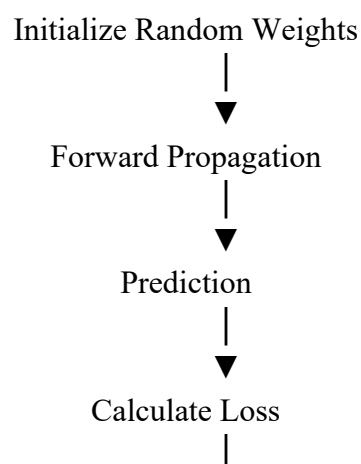
$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial J}{\partial w}$$

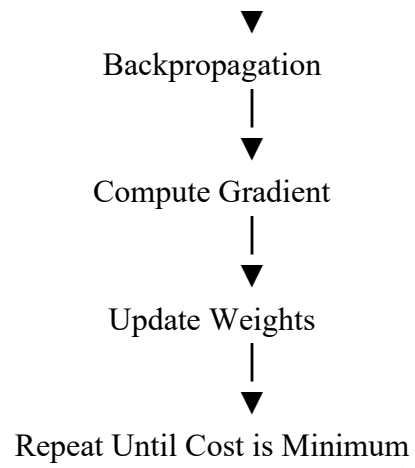
If the gradient is positive, subtracting it decreases the weight, helping reduce the cost.

- If the gradient is negative, subtracting a negative value increases the weight, which also helps reduce the cost.

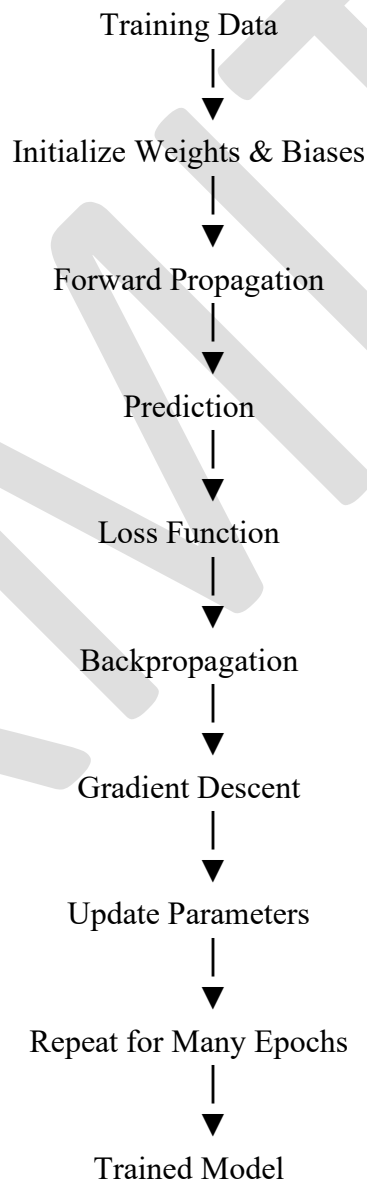
Thus, subtracting the gradient always moves the weight toward the minimum of the cost function.

### Complete Learning Cycle





#### 14. Complete Training Workflow



## Key Formula Summary

### Weighted Sum

$$z = \sum_{i=1}^n w_i x_i + b$$

### Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

### Binary Cross-Entropy Loss

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

### Gradient Descent Update

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial J}{\partial w}$$

## Key Points to Remember

- A loss function measures the error for one training example.
- A cost function is the average loss across the entire dataset.
- MSE is commonly used for regression tasks.
- Cross-Entropy Loss is widely used for classification tasks.
- Gradient Descent minimizes the cost by updating weights iteratively.
- The learning rate controls the step size during optimization.
- Mini-Batch Gradient Descent is the most commonly used optimization approach in practice.
- Backpropagation computes gradients and updates weights to reduce prediction error.

## Viva Questions

1. What is the difference between a loss function and a cost function?
2. Explain Mean Squared Error (MSE).
3. What is Cross-Entropy Loss, and when is it used?
4. Define Gradient Descent.
5. What is the purpose of the learning rate?

6. Differentiate Batch, Stochastic, and Mini-Batch Gradient Descent.
7. Explain Backpropagation.
8. Why is Backpropagation important in neural networks?
9. State the Gradient Descent weight update equation.
10. Why is Mini-Batch Gradient Descent preferred in deep learning?

Khan

## SESSION 5: Deep Learning Frameworks and Complete Deep Learning Workflow

### Learning Objectives

- Define a Deep Learning Framework.
- Explain why frameworks are essential in modern AI development.
- Understand the architecture and features of TensorFlow, Keras, PyTorch, and JAX.
- Compare these frameworks based on performance, usability, and applications.
- Understand the complete Deep Learning workflow from data collection to deployment.
- Identify real-world applications and best practices.

### 1. Introduction to Deep Learning Frameworks

#### What is a Deep Learning Framework?

A Deep Learning Framework is a software library that provides tools, functions, and pre-built modules for designing, training, testing, and deploying deep learning models.

Instead of writing complex mathematical operations from scratch, frameworks provide high-level APIs that simplify model development.

#### Definition

**A Deep Learning Framework is a collection of libraries and tools that enables developers and researchers to build, train, optimize, and deploy neural network models efficiently.**

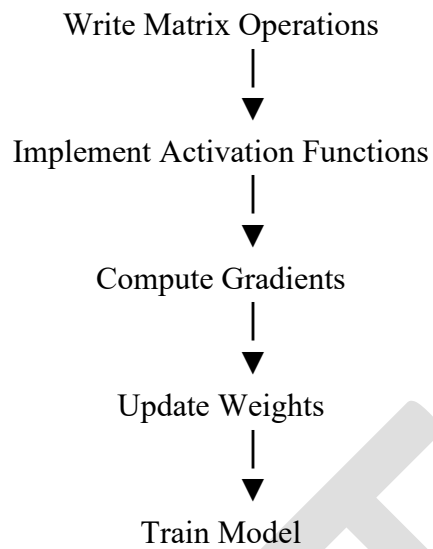
#### Why Do We Need Deep Learning Frameworks?

Without frameworks, developers would have to manually implement:

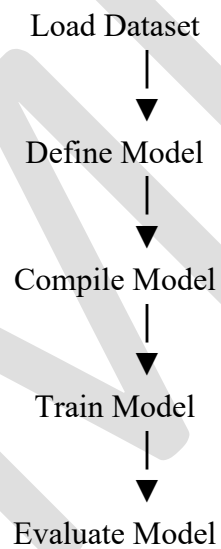
- Matrix multiplication
- Forward propagation
- Backpropagation
- Gradient computation
- Weight updates
- GPU programming
- Memory management

Frameworks automate these tasks, making development faster and less error-prone.

## Manual Approach



## Using a Framework



## 2. Features of a Deep Learning Framework

A good framework provides:

- Automatic Differentiation
- GPU/TPU Support
- Built-in Layers
- Optimizers
- Loss Functions
- Data Loaders
- Model Visualization

- Distributed Training
- Deployment Tools
- Community Support

### 3. TensorFlow

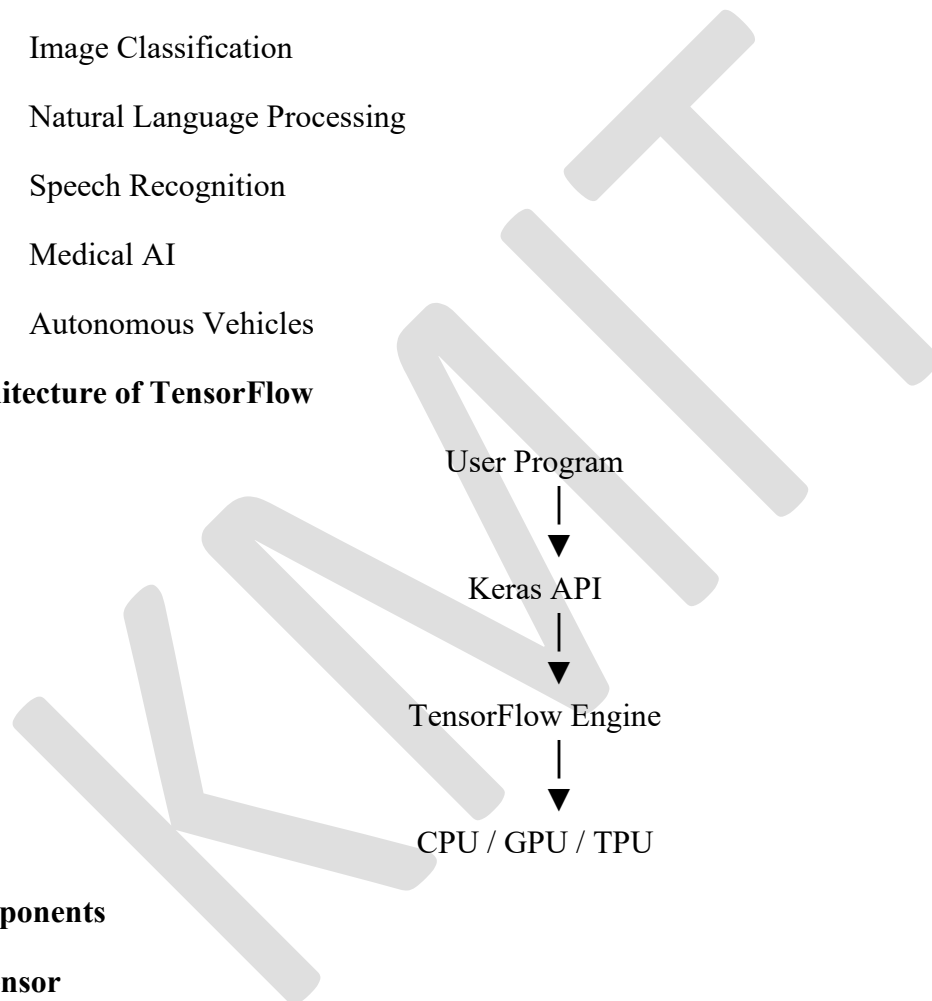
#### Introduction

TensorFlow is an open-source deep learning framework developed by Google in 2015.

It is widely used for:

- Image Classification
- Natural Language Processing
- Speech Recognition
- Medical AI
- Autonomous Vehicles

#### Architecture of TensorFlow



#### Components

##### 1. Tensor

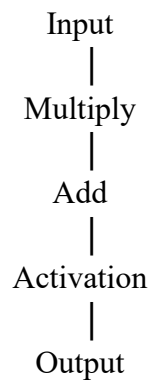
A tensor is the basic data structure in TensorFlow.

Examples:

- Scalar  $\rightarrow$  Rank 0
- Vector  $\rightarrow$  Rank 1
- Matrix  $\rightarrow$  Rank 2
- 3D Array  $\rightarrow$  Rank 3

## 2. Graph

TensorFlow represents computations as computational graphs.



## 3. Automatic Differentiation

TensorFlow automatically computes gradients during backpropagation.

No manual derivative calculations are required.

### Advantages

- Excellent production support
- Fast execution
- GPU/TPU acceleration
- Large community
- Cross-platform deployment

### Disadvantages

- Can be complex for beginners
- Larger codebase than Keras

### Applications

- Google Translate
- Google Photos
- Google Assistant
- Medical Imaging
- Self-driving Cars

## 4. Keras

### Introduction

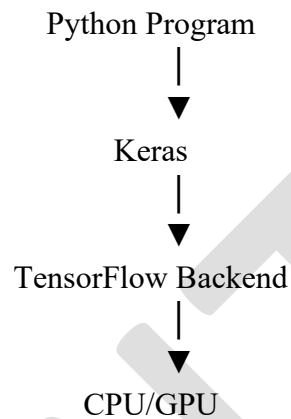
Keras is a high-level API built on top of TensorFlow.

It is designed to simplify deep learning model development.

### Why Keras?

Instead of writing hundreds of lines of code, Keras allows models to be built in just a few lines.

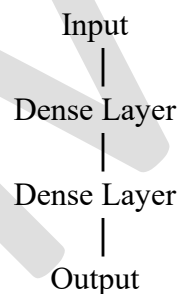
### Architecture



### Keras APIs

#### 1. Sequential API

Used for simple, layer-by-layer models.



#### 2. Functional API

Used for complex architectures with multiple inputs or outputs.

### Advantages

- Easy to learn
- Minimal code
- Excellent documentation
- Fast prototyping

### Disadvantages

- Less flexible for very complex research models

## Applications

- Academic projects
- Image classification
- Student learning
- Rapid prototyping

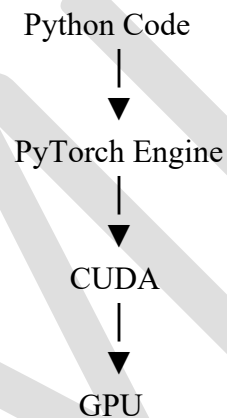
## 5. PyTorch

### Introduction

PyTorch is an open-source deep learning framework developed by **Facebook (Meta)**.

It is widely used in research due to its flexibility and dynamic computation graph.

### Architecture



### Key Components

#### Tensor

PyTorch tensors are similar to NumPy arrays but support GPU acceleration.

#### Autograd

Automatically computes gradients required for backpropagation.

#### Dynamic Computation Graph

Unlike static graphs, PyTorch builds the graph during execution.

This makes debugging easier and is preferred in research.

### Advantages

- Pythonic syntax
- Easy debugging

- Dynamic graphs
- Strong research community

### Disadvantages

- Historically weaker deployment tools than TensorFlow (though this has improved significantly)

### Applications

- AI research
- Robotics
- Computer Vision
- Natural Language Processing

## 6. JAX

### Introduction

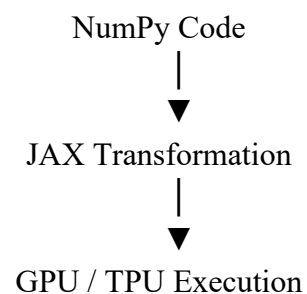
JAX is a high-performance numerical computing library developed by **Google**.

It combines the ease of NumPy with automatic differentiation and compiler optimizations.

### Features

- Automatic Differentiation
- Just-In-Time (JIT) Compilation
- Vectorization
- Parallel Computing
- TPU Support

### Workflow



### Advantages

- Extremely fast for scientific computing
- Ideal for cutting-edge AI research

- Efficient on TPUs

### Disadvantages

- Smaller ecosystem than TensorFlow or PyTorch
- Steeper learning curve

### Applications

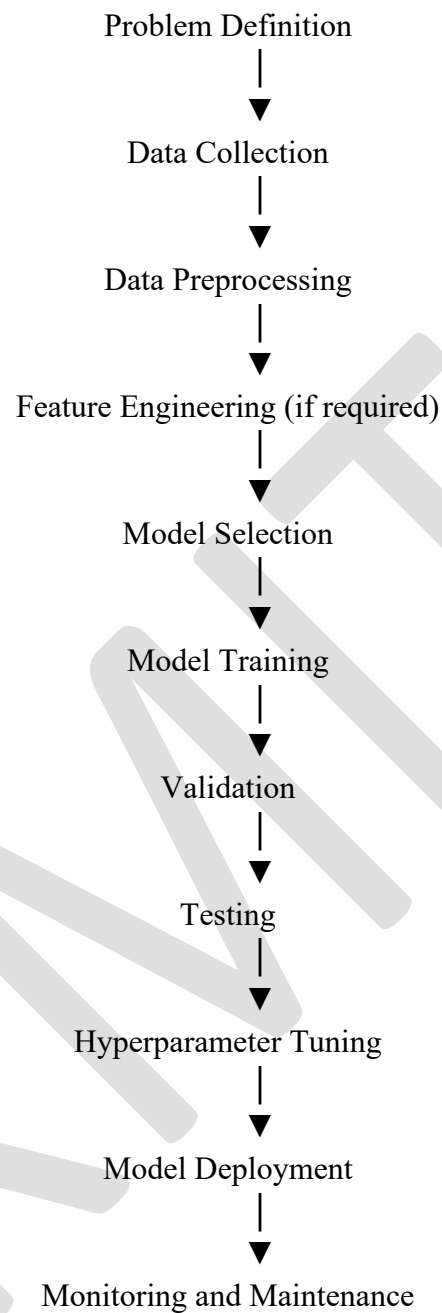
- Large Language Models (LLMs)
- Scientific simulations
- Reinforcement Learning
- Advanced AI research

## 7. Comparison of Deep Learning Frameworks

| Feature                   | TensorFlow       | Keras           | PyTorch   | JAX                  |
|---------------------------|------------------|-----------------|-----------|----------------------|
| Developed By              | Google           | Google          | Meta      | Google               |
| Ease of Learning          | Medium           | Very Easy       | Easy      | Medium               |
| Dynamic Graph             | Yes (eager mode) | Uses TensorFlow | Yes       | Yes                  |
| Automatic Differentiation | Yes              | Yes             | Yes       | Yes                  |
| GPU Support               | Excellent        | Excellent       | Excellent | Excellent            |
| TPU Support               | Excellent        | Excellent       | Limited   | Excellent            |
| Research Use              | High             | Medium          | Very High | Very High            |
| Production Deployment     | Excellent        | Excellent       | High      | Growing              |
| Best For                  | Enterprise AI    | Beginners       | Research  | Scientific Computing |

## 8. Complete Deep Learning Workflow

Developing a deep learning model follows several stages.



### Step 1: Problem Definition

Clearly identify the task.

Examples:

- Predict house prices
- Detect tumors in MRI images
- Recognize handwritten digits

- Translate languages

## **Step 2: Data Collection**

Collect relevant and high-quality data.

Sources:

- Public datasets (e.g., MNIST, CIFAR-10)
- Sensors
- Databases
- Web scraping
- User-generated data

## **Step 3: Data Preprocessing**

Prepare the data for training.

Common tasks:

- Cleaning missing values
- Removing duplicates
- Resizing images
- Normalization
- Data augmentation
- Splitting into training, validation, and test sets

## **Step 4: Feature Engineering**

For many deep learning applications, feature extraction is learned automatically.

However, some tasks may still require domain-specific feature engineering.

## **Step 5: Model Selection**

Choose the appropriate architecture.

Examples:

- CNN for images
- RNN/LSTM for sequences
- Transformer for NLP
- Autoencoder for anomaly detection

## **Step 6: Model Training**

Train the model using:

- Training data
- Loss function
- Optimizer (e.g., Adam, SGD)
- Multiple epochs

## **Step 7: Validation**

Use a validation set to monitor performance and tune hyperparameters without touching the test data.

## **Step 8: Testing**

Evaluate the final model on unseen data.

Metrics depend on the task:

- Accuracy
- Precision
- Recall
- F1-score
- Mean Squared Error (MSE)

## **Step 9: Hyperparameter Tuning**

Optimize settings such as:

- Learning rate
- Batch size
- Number of layers
- Number of neurons
- Dropout rate

## **Step 10: Deployment**

Deploy the trained model to real-world environments:

- Web applications
- Mobile apps

- Cloud services
- Embedded devices

## **Step 11: Monitoring and Maintenance**

After deployment:

- Monitor performance.
- Detect data drift.
- Retrain the model with new data when needed.
- Update the model to maintain accuracy.

## **9. Real-World Case Study: Handwritten Digit Recognition**

**Problem:** Recognize handwritten digits (0–9).

1. Collect the MNIST dataset.
2. Preprocess the images (normalize pixel values).
3. Split into training, validation, and test sets.
4. Build a neural network using Keras.
5. Train the model using Cross-Entropy Loss and the Adam optimizer.
6. Validate the model to tune hyperparameters.
7. Test on unseen images.
8. Deploy the model as a digit recognition application.

## **10. Advantages of Deep Learning Frameworks**

- Simplify model development.
- Automatic differentiation.
- Efficient GPU/TPU utilization.
- Large collection of pre-built layers and optimizers.
- Faster experimentation.
- Easier deployment and scalability.
- Strong community support and documentation.

## **11. Limitations**

- High hardware requirements.

- Steep learning curve for advanced features.
- Large memory consumption for deep models.
- Framework updates may introduce compatibility issues.
- Different frameworks may have different APIs.

### Key Points to Remember

- Deep learning frameworks provide ready-made tools for building and training neural networks.
- TensorFlow is widely used in production environments.
- Keras is ideal for beginners and rapid prototyping.
- PyTorch is popular in research due to its dynamic computation graph.
- JAX excels in scientific computing and advanced AI research.
- A successful deep learning project follows a structured workflow from problem definition to deployment and monitoring.

### Viva Questions

1. What is a deep learning framework?
2. Why are frameworks used instead of writing code from scratch?
3. Compare TensorFlow and PyTorch.
4. What is the relationship between TensorFlow and Keras?
5. What is automatic differentiation?
6. What is a tensor?
7. Explain the complete deep learning workflow.
8. Why is validation necessary?
9. What is hyperparameter tuning?
10. Why is model monitoring important after deployment?